



LAB REPORT

Course Code : ICE 3146 **Course Title :** Microprocessor and Interfacing Laboratory
Lab Task No. : 10

Lab Task Topic: Implementation of Loop-Based Display Patterns on a DOT Matrix Display.

No.	Evaluation Criteria with Marks	Put Tick (✓) Mark					Marks Obtained	Remarks
		Excellent	Good	Fair	Poor	Fail		
REPORT WRITING								
1	Theory & Objectives (2.5)							
2	Materials & Methods (5)							
3	Data Analysis & Discussion/Diagram (10)							
4	Precautions and Conclusion (2.5)							
5	Time Management (5)							
	TOTAL							
LAB PERFORMANCE								
1	Preparation for Lab (5)							
2	Experiment and Engagement (15)							
3	Communication (5)							
	TOTAL							
25 th July, 2026 Date of Experiment		01 st August , 2026 Date of Submission					_____ Teacher’s Signature	

Semester : Summer **Year :** 2026 **Level-Term :** L2-T2 **Section :** A2

Submitted to : Sameer Khairul
Lecturer, Dept. of Information and Communication Engineering
Daffodil International University

Submitted by : Tanzim Ahamed
Reg.No. : 242-50-036
Dept. of Information and Communication Engineering
Daffodil International University

Title: Implementation of Loop-Based Display Patterns on a DOT Matrix Display.

Abstract:

This experiment was performed using the Intel 8086 Microprocessor Trainer Kit to implement multiple loop-based patterns on an 8×8 dot-matrix display. The main objective was to understand how several display patterns can be stored in memory and displayed sequentially through loop instructions. The 8255 Programmable Peripheral Interface was initialized to control the row-selection and pattern-data ports. Eight font blocks were stored in memory, and each block was scanned repeatedly before the next pattern was selected. The experiment demonstrated dot-matrix multiplexing, memory addressing, loop control, font selection, delay generation, and output-port operation using 8086 assembly language.

Introduction (Theory):

The Intel 8086 is a 16-bit microprocessor that can communicate with external hardware devices through Input/Output (I/O) ports. Devices such as LEDs, seven-segment displays, LCDs, and dot matrix modules are common output peripherals. Connecting these devices with the microprocessor is called interfacing. In assembly language, the OUT instruction is used to transfer data from a register to an output port and thereby control external hardware.

An 8×8 dot matrix display consists of 64 LEDs arranged in 8 rows and 8 columns. By turning ON selected LEDs at the intersections of rows and columns, different symbols, characters, and simple graphics can be displayed. The LEDs are not usually driven all at once. Instead, the display is operated by a scanning or multiplexing technique in which one row or one column is activated at a time while the corresponding pattern data is sent to the other lines. Because the scanning happens very quickly, the human eye sees a steady complete pattern due to persistence of vision.

In this experiment, the 8255 PPI was used to control the display. Port B was used to send pattern bytes, and Port C was used to select the active row. Each display pattern was stored as eight consecutive bytes in memory. Therefore, every font block represented one complete 8×8 pattern.

The SI register pointed to the first byte of the selected font block. The SCAN procedure displayed all eight rows, while the BH register controlled how long the pattern remained visible. After one pattern was completed, eight was added to SI to select the next font block. The BL register controlled the total number of stored patterns.

Material:

Hardware	Software
1. Intel 8086 Microprocessor Trainer Kit 2. 8×8 Dot Matrix Display Module 3. Interface Connections / Trainer Board 4. Power Supply	1. Assembly Program 2. On-board Monitor / Kit Commands 3. MASM and MDA-8086

Method:

Code:

```
CODE SEGMENT
ASSUME
CS:CODE,DS:CODE,ES:CODE,SS:CODE

PPIC_C EQU 1EH
PPIC EQU 1CH
PPIB EQU 1AH
PPIA EQU 18H

ORG 1000H

MOV AL,10000000B
OUT PPIC_C,AL

MOV AL,11111111B
OUT PPIA,AL

L1: MOV SI,OFFSET FONT1

    MOV BL,8

L3: MOV BH,30

L2: PUSH SI
    CALL SCAN
    POP SI
    DEC BH
    JNZ L2

    ADD SI,8
    DEC BL
    JNZ L3

    JMP L1

SCAN PROC NEAR
    MOV AH,00000001B

SCAN1:
    MOV AL,BYTE PTR CS:[SI]
    OUT PPIB,AL

    MOV AL,AH
    OUT PPIC,AL
    CALL TIMER

    INC SI
    CLC
    ROL AH,1
    JNC SCAN1
```

```
RET
SCAN ENDP

TIMER:
    MOV CX,300
```

```
TIMER1:
    NOP
    NOP
    NOP
    NOP
    LOOP TIMER1
    RET
```

```
FONT1: DB 11111111B
        DB 11111111B
        DB 11111111B
        DB 11111111B
        DB 11111111B
        DB 11111111B
        DB 11111111B
        DB 11111111B
```

```
FONT2: DB 01111111B
        DB 01111111B
        DB 01111111B
        DB 00000000B
        DB 01111111B
        DB 01111111B
        DB 01111111B
        DB 01111111B
        DB 11111111B
```

```
FONT3: DB 11111111B
        DB 11111111B
        DB 01111110B
        DB 00000000B
        DB 01111110B
        DB 11111111B
        DB 11111111B
        DB 11111111B
```

```
FONT4: DB 11111111B
        DB 11111111B
        DB 10011101B
        DB 01101110B
        DB 01101110B
        DB 01101110B
        DB 10110001B
        DB 11111111B
```

```
FONT5: DB 11111111B
```

DB 00000000B DB 11101111B DB 11101111B DB 11101111B DB 11101111B DB 00000000B DB 11111111B FONT6: DB 11111111B DB 11000000B DB 10110111B DB 01110111B DB 01110111B DB 10110111B DB 11000000B DB 11111111B FONT7: DB 11111111B DB 11111111B	DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B FONT8: DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B DB 11111111B CODE ENDS END
--	---

Execution (PC Mode):

1. First, the assembly language program was written in Notepad and saved as .asm file.
2. Machine code execution using CMD-

C:\MDA\8086\Asm8086>MASM.exe SUM26A18.asm

Object filename [SUM26_1.OBJ]: SUM26A18

Source listing [NUL.LST]: SUM26A18

Cross-reference [NUL.CRF]:

0 Warning Errors

0 Severe Errors

3. Then again the .ABS file is created using the CMD-

C:\MDA\8086\Asm8086>LOD186 SUM26A18

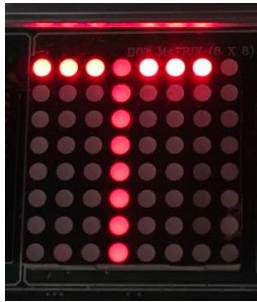
Output Object File [C:\SUM26A19.ABS]:SUM26A18

Map Filename [C:\NUL.MAP]:SUM26A18

***LOAD COMPLETED

4. Then in MDA 8086 from FILE DOWNLOAD the SUM26A18.ABS file opened and Run executed.
5. The output and register values were observed through the simulator window and the DOT Matrix Display shows the output according to the code.

Result for PC Mode:



DOT Matrix for FONT2



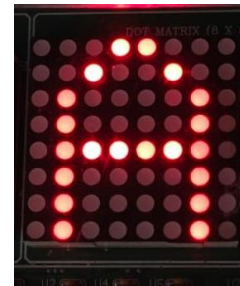
DOT Matrix for FONT3



DOT Matrix for FONT4



DOT Matrix for FONT5



DOT Matrix for FONT6

Fig 1: DOT Matrix Display outputs for different Sequence of Code.

Observed ROW/Pattern Data:

[illegible]

Discussion:

In this experiment, multiple patterns were displayed on an 8×8 dot-matrix display using loop instructions. The 8255 PPI was initialized using the control word 80H. Pattern bytes were sent to Port B, while row-selection masks were sent to Port C.

The SCAN procedure started with the row mask 00000001B. The ROL AH,1 instruction moved the active bit through all eight row positions. Therefore, the complete pattern was generated by scanning eight rows rapidly.

The BH register caused each pattern to be scanned 30 times, making it visible for a suitable duration. The SI register was increased by eight after each pattern because every font block contained eight bytes. The BL register counted the eight available font blocks. The experiment demonstrated nested-loop operation, memory-based font selection, row scanning, timing control, and continuous dot-matrix pattern generation.